

AUFGABE ZUR VORLESUNG

Übung 01 – Python in Docker

Modul	3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema	Docker: erster Container, Images, flüchtige Änderungen
Dozent	Dr. Ulrich Winkler
Semester	5. Semester · Informationstechnik

Vorlesung: Deck *docker-basics* **Ziel:** Den ersten Container starten, den Unterschied zwischen *Image* und *Container* verstehen und begreifen, warum Änderungen in einem Container **flüchtig** sind. Danach Python direkt aus einem fertigen Image ausführen.

Kontext

Ein **Image** ist eine unveränderliche Vorlage (ein „Bauplan“). Ein **Container** ist eine laufende Instanz davon. Startet man denselben Bauplan zweimal, erhält man zwei unabhängige Container. Löscht man einen Container, sind alle Änderungen *in* ihm weg – das Image bleibt unberührt. Genau dieses Prinzip erkunden Sie hier.

Öffnen Sie parallel **Portainer** (Port 9000) und beobachten Sie, wie Container auftauchen und wieder verschwinden.

Aufgabenstellung

1. Starten Sie den klassischen Test-Container `hello-world`. Lesen Sie die Ausgabe: Was hat Docker im Hintergrund getan?
2. Starten Sie einen **interaktiven** Ubuntu-Container und installieren Sie darin Python. Prüfen Sie mit `python3 --version`, dass es da ist. Verlassen Sie den Container mit `exit`.
3. Starten Sie **erneut** einen Ubuntu-Container. Ist Python noch installiert? Erklären Sie, warum (nicht).
4. Nutzen Sie stattdessen das **offizielle python-Image**: Lassen Sie sich die Python-Version anzeigen und eine kleine Rechnung ausführen – ganz ohne Installation.
5. Sehen Sie sich mit `docker ps -a` alle (auch gestoppten) Container an und räumen Sie mit `docker rm` auf. Vergleichen Sie mit der Ansicht in Portainer.

Tipp: `docker run --rm ...` löscht den Container automatisch nach dem Beenden – praktisch für kurze Experimente.

Musterlösung

```
# 1) Erster Container: Docker lädt das Image und führt es aus.
docker run hello-world

# 2) Interaktiver Ubuntu-Container, Python nachinstallieren
docker run -it ubuntu bash
    apt update && apt install -y python3          # nur IN diesem Container
    python3 --version                             # Python 3.x.y
```

```
exit

# 3) Neuer Ubuntu-Container – die Installation von eben ist WEG:
docker run -it ubuntu bash
python3 --version                # bash: python3: command not found
exit
# Grund: Punkt 2 hat nur die Schreibschicht DIESES Containers verändert.
# Der neue Container startet wieder vom unveränderten ubuntu-Image.

# 4) Offizielles Python-Image – Python ist schon drin:
docker run --rm python:3.13-slim python3 --version
docker run --rm python:3.13-slim python3 -c "print(6 * 7)"    # 42

# 5) Aufräumen
docker ps -a                # zeigt auch gestoppte Container
docker rm <container-id>    # einzeln entfernen (IDs aus der Liste)
docker container prune      # alle gestoppten auf einmal
```

Was Sie gelernt haben: Image ≠ Container; jeder `docker run` erzeugt eine frische Instanz aus dem Image; Änderungen in einem Container sind flüchtig; für Python nimmt man ein fertiges Image statt es mühsam zu installieren.